

写给不懂编程的人 · 精简版

AgentLoom 是怎么搭起来的

十二页读完：这个系统由哪几块组成、各自解决什么问题、为什么当初这么选。每个技术名词第一次出现都用大白话解释，不预设任何编程背景。

想看更细的展开，见同目录《详解》(67 页)。本册是主文档，详解是附录。

第 1 节

这个系统在干什么

设想一个任务：每周一早上自动生成项目周报——去代码仓库拉上周提交、去任务系统拉已完成条目、把两边对上、写成人话、发邮件。

你可能会问：这不就是把资料丢给 AI 让它写吗？问题在于这件事有五个步骤，而且每步都可能出岔子。全塞进一次对话，会撞上三个麻烦：

- 不知道错在哪。周报少了一个项目，是仓库没拉到、AI 对不上、还是写漏了？一次对话是黑盒，只能重来。
- 没法只重跑一步。第一步拉了三分钟数据，第四步崩了，得从头再等一遍。
- 有些步骤不该交给 AI。「发邮件」要的是按模板发给指定地址，不是让 AI 决定收件人。

所以 AgentLoom 让你把步骤画出来：每步是一个方块，方块用线连起来表示先后顺序。点运行，系统按顺序执行，**每个方块的状态、输入、输出都单独记录**。出错时你能看到「第三个方块失败了，它当时收到的是这些数据」。

工作流 Workflow

就是你画的那张图，加上每个方块的配置。它描述「这件事分几步、每步干什么、谁在谁之后」，本身不会自己动，要有人点运行。

技术上这叫「有向无环图」：有向是线有箭头（B 要等 A 干完），无环是不许绕回去成圈（否则大家互相等，永远卡住）。可以直接理解成菜谱的步骤表——切菜在炒菜前，不可能反过来。

智能体 Agent

一个有持续身份的 AI 角色，跟「问一次 AI」的区别在四点：有自己的工具（能读文件、跑命令）、能多轮自主行动（自己决定下一步用哪个工具）、有记忆（上次的结论下次还记得）、有版本（改了配置旧版本还在）。

注意：工作流和智能体是两个平级的概念，不是一个套在另一个里。你可以只用智能体不画工作流，也可以画一条工作流里面一个 AI 都没有。智能体能塞进工作流当方块用，只是其中一种连接方式。

所以系统的核心职责是四件事

职责	难在哪
让你把步骤画出来	要在你 连线的时候 就拦住不合法的连法，而不是等运行时才报错
按顺序执行	要处理分支、循环、嵌套，挂掉时能说清挂在哪一步
让 AI 真能干活	AI 要能跑命令、改文件，这意味着要给它一台真机器，还得保证它跑不出去
让多个组织同时用	数据不能串、资源要有上限、行为要留痕

顺手建立一套对应关系：把它想象成一间餐厅

这一节给你一套对应关系，后面每节都可以回来对一眼。

整体对应

顾客在电子菜单上[自己设计一道菜的做法](#)（画工作流），下单。前台接单并检查能不能做（服务器）。单子不直接进后厨，而是[挂到叫号板上](#)（队列）。厨师按板上顺序取单开工（工作进程），要动火的工序在[独立封闭灶台](#)里做（沙箱）。食材分放[四个仓库](#)（四种数据库）。整间餐厅同时服务多个包间，[包间之间看不到彼此的单子](#)（多租户隔离）。

餐厅里的东西	系统里对应	本册第几节
电子菜单，拖拽设计做法	画布	第 2 节
前台，接单验单	服务器	第 2 节
叫号板，单子先挂上去	队列	第 2 节
封闭灶台，一人一个	沙箱	第 2、6 节
四个仓库，分门别类	四种数据库	第 2 节
包间，A 桌看不到 B 桌	多租户隔离	第 6 节
后厨统一术语手册	契约层	第 3、6 节

类比的失效之处：厨师不会自己改菜谱，但智能体能提议改自己的配置（经你批准）；餐厅灶台是固定的，沙箱用完就整个销毁。

第 2 节

五个组成部分

一、画布：你唯一看得见的部分

职责听起来简单——让人拖方块连线——但有个要求让它变难：不合法的连法必须在你松手之前就被拦住。

不这样会出什么事

你画了二十个方块，点运行，跑到第十七个才报「数据类型不对」。此时前十六步已花掉几分钟、还可能调用了付费的 AI 接口。改完再跑又要从头等。检查越早，代价越低——这是全系统反复出现的一条原则。

怎么判断能不能连：每个方块的输入输出口都有类型标签（共 14 种，如文本、图片、模型、工具、沙箱）。就像插头和插座，两孔插不进三孔，这件事不用真的通电就能确定。但实际比插头复杂，因为有些类型之间能转换，所以结论分四档：完全兼容（直接连）、需要转换（中间插一步，只允许三种）、部分兼容（字段对不齐，系统按字段名相似度猜你想怎么对应）、不兼容（拒绝并说明原因）。

「需要转换」只允许三种是有意的：每多一条自动转换，用户就多一次「它居然偷偷帮我转了」的意外。

二、服务器：接单和派活，但不干重活

任何请求进来要过五道关，顺序固定：初步解析 → 限流 → 验证身份 → 组织准入 → 角色权限。全过之后，系统把接下来的数据库操作包进一个事务，并在事务里设好「当前是哪个组织」。

它不干重活，因为一次工作流可能跑几十分钟。如果接到请求就开始干、干完才回复，会出三个问题：浏览器等不了那么久会超时；处理能力被占满，同时来十个长任务第十一个人连页面都打不开；服务器一重启正在跑的活全丢，因为只存在于内存里。

所以做法是：把意图写进数据库，往队列放一张单子，立刻回复「收到了，编号 XXX」。

三、队列：为什么不能「来了就干」

打个比方

点单后单子不直接塞给某个厨师，而是打印出来挂叫号板。好处三个：哪个厨师空了自己来取，不会一个忙死一个闲着；某厨师突然请假，他没做完的单子还在板上，别人能接手；单上记着做了几次，反复失败就不再重发，转人工。

对应三个技术能力：削峰（过载时变慢而不是崩——变慢可恢复，崩了要人工介入）、可靠（进程崩了单子还在队列里，超时后重新分配）、定时与重试（重试间隔递增，避免把已经挂了的服务压得更死）。这里有个顺序不能错：必须先写完数据库、确认提交成功，才往队列放单子——反了的话工作进程可能在数据库写完前就取到单子、发现记录不存在，而这种 bug 的表现是随机的（机器快时没事、慢时报错），极难排查。

四、沙箱：为什么不敢让 AI 直接在服务器上跑

AI 生成的代码是不可信输入。这跟「AI 好不好」无关，是个结构性事实：代码是现场生成的，没人逐行审过就执行了。风险不一定来自恶意，更常见的是路径写错的删除命令、写满磁盘的循环、不小心读到密钥。

容器与微型虚拟机 Container / Firecracker

容器（也就是常说的 Docker）是主流的轻量隔离方式，启动快、开销小。但它有个根本限制：所有容器共享宿主机的操作系统内核。内核有漏洞，容器里的程序就有可能突破边界。

微型虚拟机让每个沙箱有自己独立的内核，那条突破路径就被堵住了。传统虚拟机笨重（启动几十秒），Firecracker 是专为「大量、短命、快启动」做的，启动在毫秒级。代价是要自己管生命周期、网络、磁盘，比敲一行 `docker run` 复杂一个量级——这部分复杂度由一个 Go 写的组件承担。

打个比方

容器像同一栋楼的不同房间——隔音、有门锁，但共用水电和承重结构。微型虚拟机像各自独立的小屋，自带水电。跑自己审过的代码用房间够了；跑不知道会干什么的代码，值得盖独立小屋。

五、四种数据库：为什么不是一种

四类数据性质完全不同，只用一种会在四个地方各撞一次墙：

存什么	用哪个	只用主数据库会怎样
业务记录	PostgreSQL	这就是它的主场
队列、计数器	Redis（存内存里，极快）	每次取单都读写磁盘，几百次每秒就把主库拖垮，而主库一慢全系统都慢
按意思检索	Qdrant（向量数据库）	小规模够用，量大之后检索性能和调优手段不如专用的
文件	MinIO（自建网盘）	大字段撑爆数据表、拖慢备份，每次全表操作都变贵

代价：四套都要部署监控备份；跨库没有事务（往数据库写记录和往文件存储上传不能保证同生共死）；新人本地开发要起四个服务。

一个容易搞混的点：智能体的记忆不用向量数据库，它是一张存在 PostgreSQL 里的图，检索用数据库自带的全文检索。向量数据库用在知识库检索和智能路由的学习上。原因是记忆由系统自己写、用词规范，关键词匹配够用，还省掉一次把文字转成向量的 AI 调用；而知识库装用户上传的文档，用户问「怎么上线」文档写「部署流程」，没有共同的词，只有按意思检索能对上。

第 3 节

Zod 是什么，为什么它这么重要

这一节给的篇幅最多，因为 Zod 在这个项目里不只是一个工具库，它是整套跨端约定的地基。

先讲一个前提：类型检查在运行时消失

TypeScript

给 JavaScript 加类型检查的版本。你声明「这个参数必须是数字」，然后有个工具在真正运行之前先核对一遍，对不上就报错——错误被提前到写代码的时候，你在编辑器里就能看到红线。

但关键在于：它检查完之后，所有类型信息就被删掉了，真正跑起来的还是普通 JavaScript。

这意味着：如果数据是运行时才从外面进来的——用户提交的表单、别的系统发来的请求——TypeScript 一点忙都帮不上。你可以声明「我期望收到一个有 name 字段的对象」，但对方实际发来一段乱码，程序照样接受，然后在某个地方崩掉。

打个比方

TypeScript 像装修时的图纸审查——能确保设计上门宽匹配门框。但它管不了[装修完成之后谁从门进来](#)。要在门口验证来客，需要另一个东西：门卫。Zod 就是那个门卫。

手写门卫的问题

服务器收到创建智能体的请求，得自己检查。手写版大概这样：

```
if (typeof body.name !== 'string' || body.name.length === 0) {
  throw new Error('name 必须是非空文字');
}
if (body.runtimeMode !== 'sandbox' && body.runtimeMode !== 'no_sandbox') {
  throw new Error('runtimeMode 取值不对');
}
// .....每个字段都要来一遍
```

三个问题：啰嗦（几十个接口每个都写一大段）、容易漏（加了新字段忘了加检查），以及最要命的一个：

```
type CreateAgent = { name: string; runtimeMode: 'sandbox' | 'no_sandbox' };
// 类型声明说 runtimeMode 有两种取值

if (body.runtimeMode !== 'sandbox') { throw ... }
// 检查代码却只认一种。两边对不上，没有任何工具会提醒你
```

不这样会出什么事

类型声明和实际检查各写一份，就一定会在某次修改后不一致。而且这种不一致没有任何工具能自动发现——类型检查器只看声明，测试只测检查逻辑，两边都觉得自己是对的。最终表现为线上偶发的奇怪数据。

Zod 的做法：只写一份，类型自动推导出来

```
const CreateAgentSchema = z.object({
  name: z.string().min(1),
  runtimeMode: z.enum(['sandbox', 'no_sandbox']),
});
```

这一份东西同时给你两样：

```
// 一、运行时的检查器
CreateAgentSchema.parse(收到的数据)
// 合规就返回数据；不合规就抛错，并说明哪个字段错在哪

// 二、静态类型，从上面那份规格「反推」出来
type CreateAgent = z.infer<typeof CreateAgentSchema>;
// 等价于 { name: string; runtimeMode: 'sandbox' | 'no_sandbox' }
```

这里是关键：类型不是你另写的，是从检查器身上推导出来的。所以「类型说三种、检查只认两种」这种事在结构上就不可能发生——它们本来就是同一个东西的两个面。

打个比方

过去是：门口贴一张「入场要求」海报（类型声明），门卫另有一本自己的检查手册（校验代码）。海报改了没人通知门卫，于是海报写「凭票入场」，门卫还在只看身份证。

Zod 的做法是：只有门卫的手册是真的，海报直接由手册自动打印。手册改一个字，海报下一秒跟着变。

项目里怎么用

规格写好后包装成框架认识的形式，再挂到接收请求的地方：

```
async create(  
  @Body(new ZodValidationPipe(CreateAgentDefinitionSchema))  
  dto: CreateAgentDefinitionDto,  
) { ... }
```

意思是：这个接口的请求体，进来之前先用那份规格检查。通过了 `dto` 就是干净可信的数据；没通过，请求根本进不到函数体里。检查失败返回 422，附一个 `errors` 数组，每条是「哪个字段、错在哪」，前端可以直接显示在对应输入框下面。

一个反直觉的坑：请求和响应不能共用一份规格

Zod 支持默认值。分页查询里 `page: ....default(1)` 意思是「客户端可以不传，不传当 1」。这对请求完全正确。但服务器返回响应时，语义反过来了：

场景	page 字段的语义
请求	可选——客户端不传也行
响应	必填——服务器一定会给

图省事直接拿请求规格当响应用，接口文档生成器会按请求的语义理解，把字段标成「可能没有」。于是前端得写一堆没必要的判空；更糟的是凭空放宽了真实的接口约定——明明服务器保证一定给，文档却说可能没有。

所以项目的规矩是：响应必须另写一份规格，按「输出形状」建模。

为什么不用同类的 class-validator

那个方案是在类的属性上加标记，看起来更清爽，但 `@IsString()` 和 `name: string` 是两份独立的声明，可以互相矛盾且没人报错。这就回到本节开头那个问题。Zod 用「类型从校验器推导」在结构上消灭了这个可能。

第 4 节

其余技术为什么这么选

每一条的理由都不是「这个技术好」，而是「不用它会踩某个具体的坑」。

技术	干什么	为什么是它
NestJS	服务器代码怎么组织	41 个业务模块不用框架很快会变成一团麻。它强制统一结构，价值不在多聪明而在统一——任何人打开任何模块都知道去哪找什么
Fastify	收发网络请求	替换掉框架默认的 Express，理由是吞吐更高，代价是部分 Express 插件不能直接用
Drizzle	操作数据库，不用手写 SQL	手拼 SQL 字符串会被 注入攻击 ，字段名写错还要等运行时才发现。选它而非更流行的 Prisma，因为它薄——代码和生成的 SQL 几乎一一对应，性能可预判
BullMQ	队列（叫号板）	见第 2 节，顺带解决定时任务与失败重试
Socket.IO	服务器主动推消息	普通交互是「网页问一句服务器答一句」，服务器没法主动说话。有了它执行进度能实时推到屏幕上，断线重连还能补上错过的消息
React + React Flow	网页界面 + 拖拽画布	画布类库都在 React 生态里，拖拽连线缩放都是现成的
Flutter	手机端	一套代码同时生成 iOS 和安卓，不用把界面写两遍
Go	管理沙箱虚拟机	启动虚拟机、配网卡、下发防火墙规则都是贴着操作系统的活。用 JavaScript 要不停调外部命令、解析文字输出，很别扭；Go 有成熟的库直接操作

前端有一条明确的分工线

这条规矩值得单独说，因为它防的是一类很烦的 bug：

数据来源	交给谁管	例子
从服务器拿来的	TanStack Query	工作流列表、执行历史
纯本地、临时的	Zustand	画布上选中了哪个方块
该出现在网址里的	URL 参数	筛选条件、当前页码

不这样会出什么事

把服务器数据也塞进本地状态管理，就出现两份真相：一份在本地一份在服务器。谁覆盖谁会变成一团糊，典型症状是「我明明改了，刷新一下又变回去了」。

筛选条件放网址的理由更实际：用户能把当前筛选结果的链接发给同事。

为什么会掺进 Rust

Rust 负责第 2 节讲的那个连线判断，编译成一种能在浏览器里跑的接近机器码的格式（叫 WebAssembly，简称 WASM），放在后台线程执行——因为这个判断在你拖鼠标时会被反复触发，占用主线程画布会卡。

当初选它的理由有两条：一是递归结构比较这类算法用 Rust 表达最清晰、编译产物小；二是同一份规则服务器也能用，避免前后端各写一套然后慢慢跑偏。

第二条没兑现，这点必须说清

经核实：服务器端没有任何地方调用这个判断，它只在网页端加载。而且 Rust 那边其实写了一个更严格的版本（多了「方向必须输出接输入」「可选的源不能接必填的目标」等检查），但它没有对外暴露接口，浏览器根本调不到。实际生效的是网页端另写的一份宽松版。

于是出现了两份规则：严格的没在用，宽松的在用——正是当初想避免的「各写一套然后跑偏」，只是跑偏形式变了。诚实的结论是：按现状看，选 Rust 这件事偏重了。

顺带提醒：如果有人问「为什么不用 TypeScript 写」，正确答案是「为了不写两遍」而不是「为了快」。Rust 和 JavaScript 之间传数据要来回转格式，数据量小时这开销可能比计算本身还大；答「为了快」会被追问性能对比数据，而这个项目没做过那种实验。

第 5 节

点一次「运行」发生了什么

把前面所有零件按时间顺序串起来。如果只想快速理解整个系统，读这一节就够。

阶段一：请求进门（一百毫秒以内）

1. 浏览器发出请求，带上身份凭据。
2. 过五道关：初步解析、限流、验证身份、组织准入、角色权限。任一关不通过就在这里被拒，你会看到明确的错误。
3. 检查请求内容，用第 3 节讲的 Zod 规格。不合规返回 422 并指出哪个字段错了。
4. 开事务、设租户上下文——告诉数据库「接下来的操作属于这个组织」。
5. 写执行记录，状态「排队中」，并把当前画布存一份快照。
6. 提交事务，然后放队列。顺序不能反，原因见第 2 节。
7. 立刻回复「收到了，编号 XXX」。请求到这里就结束了，真正的活还没开始。

为什么第 5 步要存快照

因为你可能在任务跑的过程中继续编辑画布。如果执行时去读「当前的画布」，就会出现跑到一半规则变了的荒唐情况。存快照等于把单子内容当场复印一份——之后你怎么改菜单都不影响这张已下的单。

阶段二：网页开始盯着（并行发生）

拿到编号后，网页订阅这个执行的进度，订阅时带上「我已收到第几号消息」（首次是 0）。服务器先回一份当前整体快照——因为从你点运行到订阅成功之间可能已经产生了事件。

阶段三：真正干活（几秒到几十分钟）

某个空闲的工作进程取到单子，状态改成「运行中」，推一条事件出去——你的屏幕开始有反应。接着调度器读快照里的图，算出「哪些方块现在可以跑」（所有前置都已完成的那些），然后循环：

1. 收集输入——从上游方块的输出里取，按连线关系装配
2. 按方块类型干活——调 AI、查知识库、执行代码、发请求
3. 记录结果——输出存进数据库，状态改「已完成」
4. 推事件——你屏幕上那个方块变绿
5. 算出下一批能跑的方块，回到第 1 步

如果方块里是 AI 且要执行命令，还要先起一个沙箱，把 AI 的工具调用转发进去执行，再把结果拿回来。

遇到需要授权的操作会挂起。AI 想写文件或改配置时，执行暂停、你的屏幕弹确认框。**此时任务不是失败，只是在等。**你一直不理就按超时策略处理——自动同意、自动拒绝、或升级给上级（最多三次）。

阶段四：中途可能出的岔子

出了什么事	系统怎么处理
某个方块失败	按配置重试若干次、间隔递增。仍失败则标记失败，并按配置决定整条终止还是走失败分支
你的网页断了	重连时带上「我已收到第 15 号」，服务器只补第 16 号之后的，不重放全部。所以刷新页面不丢进度
工作进程崩了	单子还在队列里，超时后重新分配。已完成的方块不会重跑，因为状态和输出都已落库

阶段五：收尾

加密敏感产物（AI 输出等，在写库之前加密）、写审计日志（记进一张只能追加的表）、发通知（站内、邮件或手机推送）、回收沙箱（会话型直接销毁；持久型只停机保留磁盘，下次可重启接着用）。

加密到底是怎么做的 客户端持钥加密

这套机制常被叫作「端到端加密」，但它跟聊天软件那种端到端加密不是一回事，讲错了会被追问。真实的流程是三步：

一、你在浏览器里生成一对密钥，**只把公钥上传给服务器**，私钥留在本地（存在浏览器的本地存储里，也可以导出成文件自己保管）。二、AI 的输出在服务器的进程里产生，服务器**用你的公钥**把它加密，然后才写进数据库。三、你要查看时，浏览器用本地私钥解开。

所以准确的说法是「**客户端持钥的静态数据加密**」：钥匙确实只在你手里，数据库拖走也读不出；**但明文在服务器的进程里确实存在过那一瞬间**。它防的是「数据库被偷、备份泄露、运维越权查表」，不防「服务器运行时被攻破」。用的算法是 RSA-4096 包一把一次性的 AES-256 密钥。

整条链的共同特征

回头看，有一个模式贯穿全程：每一步都先落库，再往下走。

成本很明显——数据库写入次数多了很多，整体速度不如「全在内存里算完再存一次」。换来两个能力：任何一步崩掉都能说清崩在哪、当时输入是什么；崩了能从断点继续，不用整条重跑。

这是典型的取舍：**用性能换可观测性和可恢复性**。对一个动辄跑几十分钟、还要花钱调 AI 的系统，这笔交换划算。如果是个毫秒级的接口，就不划算了。

第 6 节

三处最费心的设计

这三个问题的共同特点是：错了不会当场报错，而是半年后变成事故。

一、怎么保证三个端说同一种话

系统有三个端（网页、手机、服务器），它们之间传数据要有约定。典型的事故是这样：服务器实现是「用户没填描述就返回空值」，网页开发者以为「这个字段一定有内容」直接拿来显示。绝大多数数据都填了，一切正常；某天有人建了条没填的，页面白屏。

这类问题为什么特别难防

不是立刻发生（可能上线三个月后才第一次触发）、不是必然发生（取决于数据，测试环境往往碰不到）、没有任何工具会警告你——服务器代码是对的，网页代码单独看也是对的，错的是两者之间那份没人维护的默契。

最直觉的解法是「写份接口文档让大家照着实现」，但它失效得很快——文档和代码是两个东西，改了一个不会强迫你改另一个。三个月后文档开始说谎，而且没人知道它从哪句开始说谎。

这个项目的解法是：让定义只有一份，其余全自动生成。服务器用 Zod 写规格（唯一真相）→ 自动生成机器可读的接口文档 → 自动生成一份 TypeScript 类型文件 → 网页端引用它，不许自己另写。

结果是：服务器改一个字段，重新生成之后，网页端所有用到它的地方立刻编译报错。错误在几秒内暴露，而不是三个月后白屏。

为什么这比「写个测试检查一致性」更强

测试可以被跳过——赶进度时加个跳过标记，代码照样上线。而生成方案里一致性是由编译保证的：用了不存在的字段，编译就失败，打包产物根本产生不出来。编译不能被跳过，否则连东西都没有。

自动生成也有失灵处：画布方块的内部配置是「什么都可能有」的自由结构，接口文档标准表达不了，生成出的类型会退化成「一个啥都行的对象」。项目的处理是显式承认——在网页端把这三个字段从生成类型里摘掉换成手写的，其余仍用生成的。价值在于它是显式的，任何人看到都知道「这里特殊处理过」。

二、笼子是怎么焊住的

有了独立内核的虚拟机还不够，里面的程序仍能读写文件、发网络请求。所以四个方向各加一道锁。文件相关的检查在沙箱内部那个 Go 程序里，不在服务器上——如果在服务器做，沙箱里的程序就有机会绕过它直接碰文件系统。

方向	怎么做
读文件	只允许两个固定目录。判断时连着路径分隔符一起比（否则 <code>/workspace-evil</code> 也会通过）；然后把软链接解开再判一次——因为允许目录里的一个软链接可以指向外面，路径字符串看起来完全合法
写文件	更严：不许把根目录当文件写；已存在的目标必须是普通文件（拒绝目录、管道、设备节点）；要新建的则检查其父目录
解压缩包	独立的攻击面——包里每个条目自带路径，构造一个 <code>../..</code> 就能写到任何地方。所以逐条检查，拒绝绝对路径，拼接后必须仍在目标目录下
网络	网卡入口一串规则：普通包要求源地址和源硬件地址同时匹配分配值才放行、地址询问包同理、新版协议整类丢弃、最后一条「匹配一切即丢弃」兜底。另有防火墙规则拒绝沙箱互访与访问内网
凭据	沙箱内不持有能访问应用的证书，回调必须过一个中转站，而中转站转发前会先把请求里原有的身份信息剥掉再换上沙箱自己的令牌

四道锁实现完全不同，但有一条共同原则：默认拒绝，明确放行。不列举「不允许什么」而是列举「只允许什么」。差别在漏写一条的后果——前者漏一条是一个漏洞，后者漏一条只是少了个功能。

三、怎么保证 A 公司看不到 B 公司的数据

所有组织的数据都在同一个数据库同一张表里，靠查询条件区分。某个人写查询时忘加一个条件，这个查询就会返回所有组织的数据——代码能跑，测试可能也过（测试库往往只有一个组织的数据），直到上线后有用户看到了别人的东西。

为什么「小心一点」不是解决方案

系统有 41 个业务模块、几百处查询。要求每处都不漏写，等于把安全押在「每个人每次都记得」上。这是纪律，而纪律必然会在某个时刻失效——新人不知道这条规矩、赶进度时忘了、复制粘贴改漏了一处。

第一道锁在应用层：验证身份之后（注意是之后，之前请求里带的任何组织信息都不可信），开一个事务并在里面告诉数据库当前是哪个组织。这个设置**只在本事务内有效**，事务一结束自动失效——这点很关键，因为数据库连接是复用的，如果设置留在连接上，下一个请求可能带着上一个组织的上下文运行。

第二道锁在数据库：68 张表挂了规则，规定「这一行的组织字段必须等于当前设置的组织」。规则由数据库自己执行，**在查询语句之外**——所以即使应用发来的查询完全没有组织条件，数据库也只返回符合规则的行。

打个比方

第一道锁是**前台按包间号分发单据**，靠流程保证不送错。第二道锁是**每个包间的门自己认人**，即使前台送错了门也不会开。

第二道锁的价值恰恰在于它假设第一道会出错。如果前台永远不错，第二道就是多余的；正因为前台是代码、是纪律，第二道才有意义。

怎么证明它真生效：测试时启动一个真实的数据库容器、逐条回放全部迁移脚本把规则真建出来，然后分带/不带组织上下文两种情形查询，断言跨组织查询返回空或直接报权限错误。用真数据库而不是模拟，因为模拟出来的假数据库不会执行这些规则，测了也白测。

第 7 节

面试时怎么说自己的贡献

你的处境是：系统设计是你做的，代码基本由 AI 生成。这一节讲怎么把这件事说清楚，以及为什么「说清楚」比「含糊过去」有力得多。

为什么不能宣称「代码是我手写的」

因为面试官验证归属只有一个手段：顺着因果往下追问。他不会问「这是你写的吗」，他会问「这里为什么这么做」「不这么做会怎样」「边界在哪」。

这是个负期望的赌注

宣称范围越大，被问到自己不懂的部分的概率越高。而一旦被问穿一次，对方会重新审视你说过的每一句话——包括那些你真的懂的部分。收益（听起来做得多）远小于风险（整体不可信）。

按「失误的影响范围」分三档

比「说了多少行」更有用的分法是这个。理由是：影响范围大的地方必须由设计者亲自定边界，因为它的错误不会局限在一个模块内。

档位	包含什么
我主导设计与验收	跨端契约层、沙箱隔离边界。这两处的错误会跨语言、跨端、跨信任边界扩散
我定语义、按行为验收	多租户隔离、调度语义、事件协议、路由策略、类型引擎
生成为主、抽查为辅	41 个业务模块的增删改查、网页页面、手机端、插件市场与分账

第一档只有两处，这是核实之后的结果，不是凑不出三个。原本想把多租户放进第一档，但核实后发现它的配额部分有个没修的缺口（见本节末的清单），不满足「已验收」的标准，所以降档。这个「宁可少一处、不凑数」的表述本身有说服力——它说明你对自己的判断有标准，而不是挑好听的说。

可以直接用的一段话

「系统设计是我做的，代码大量由 AI 生成。有两处边界是我主导设计并定验收标准的——跨端契约层、沙箱隔离边界，因为这两处的失误会跨端或跨信任边界扩散，不是单模块问题。

多租户的数据面是闭环的，68 张表挂了行级安全策略，事务上下文只读验签后的身份；但配额计费那一段排在认证之前、接受未验签的租户身份，这是我自己核实出来的缺口，还没修。

这几处的实现和测试我能讲清。具体某个业务模块的实现细节，我会答不上来。」

为什么这样说比宣称手写更强

- 它是真的，所以经得起任意深度的追问
- 它划定了范围，对方知道该往哪问，你不会被问到自己不懂的地方
- 主动报出自己找到的缺口，是很强的能力信号——证明你真核实过，而不是背了一遍文档

如今的面试官普遍接受 AI 辅助编程。真正稀缺的不是「手写了多少」，而是「知道哪里必须自己把关，以及为什么」——这个判断力很难伪造，而你恰好有。

兑现成本：说了「这几处我能讲清」就必须真讲得清——面试前把第 6 节那三处的实现吃透一遍，这是可控的准备量。而生成为主那一档，提前承认答不上来，比现场被问穿代价小得多。

已知还没修的问题

下面每条都回源码核对过。声称某一块是自己的贡献之前，先确认它不在这张表上。

一个真实的安全缺口：配额可以被别人刷干

问题在两个检查的顺序：限流排在验证身份之前。限流需要知道「这是哪个组织」才能扣额度，但此时身份还没验，它只能用请求里自己声称的组织身份——也就是未经验证的。

1. 攻击者构造一个凭据，签名部分随便填，但里面写着受害者的组织 ID
2. 用它反复请求任意一个受限流的接口
3. 限流那一关读到受害者的组织 ID，把受害者今天的额度扣掉一次
4. 然后才走到验身份那一关，签名不对，返回拒绝

5. 但额度已经扣了。重复几千次，受害者当天的额度就被刷干

这个漏洞的性质

它不泄露任何数据——第 6 节那两道锁是完好的。它是一个[可用性攻击](#)：让受害者的正常请求被系统自己拒绝。

严重性在两点：不需要任何有效凭据（谁都能干），可以精确指定受害者（不是随机破坏）。

修的方向：把配额计费移到验证身份之后；或者对未验证的请求只按来源 IP 计数。修完要补一个回归测试——用签名无效的凭据打接口，断言受害者组织的额度计数不变。

其余已核实的缺口

缺口	现状
类型引擎没接到服务器	见第 4 节：服务器完全没调用它；Rust 里那份更严格的连线判断没有对外接口，浏览器调不到，实际生效的是网页端另写的宽松版
对话的断线补播没接通	工作流那边是好的。但对话那边只声明了这个字段，订阅时不上报、收到消息也不更新，等于能力没接上
一个流式适配器没在用	代码和单元测试都有，但整个服务器没有任何地方调用它
插件工具的「发布」不上传	它只签名并把包写回本地，不上传；但命令说明写的是「发布到市场」
文档说错了哪张表只能追加	文档称租户密钥表只能追加，实际不是——迁移脚本给了完整的修改删除规则。真正只能追加的是两张审计表
审计写入口不唯一	治理模块另有一条直接写审计表的路径。这是有意的——主事务会因拦截回滚，审计必须独立落地，否则「拦了但没留痕」。失真的是文档措辞「唯一正式写入口」
优化建议不能采纳	四类建议写入的位置对执行不产生实际影响，所以前后端用同一份空名单显式禁用「采纳」，只留「忽略」。这是刻意的，不是残缺

两条方法纪律

这份清单在整理过程中出过几次错，教训比清单本身更有价值。

一、别拿自己的旧笔记当现状。清单里原本有一条「某个面板没有被使用」，是从三周前的工作记录里抄来的。回源码一看，那个面板早就被正式引入并渲染了。

二、别拿测试替身当生产实现。讲沙箱文件边界时，最初引用的是一份[只在特定环境变量打开时才启用的测试用实现](#)。真正的生产实现在沙箱内部那个 Go 程序里，而且严格得多。同类的错还犯过一次：讲加密时把「浏览器里加密后再上传」当成了现状，而实际上浏览器侧只有解密能力，加密发生在服务器的工作进程里。

这几条的共同点

都是把「代码存在」当成了「代码生效」，或者把「听起来应该这样」当成了「实际这样」。确认一段代码是否在生产路径上，比读懂这段代码更重要。

判断方法很具体，三个问题问完再下结论：它有没有被调用？调用它的地方有没有被环境变量门控？它在测试目录下吗？

本册完，共九节。想看更细的展开——每个名词的完整解释、每个技术选型的详细对比、17 步而非 7 步的运行流程——见同目录《AgentLoom 是怎么搭起来的·详解》。那一份是本册的附录，结论完全一致，只是把每一节摊开讲。